

Deployable Reference Architecture: AWS ECS Fargate Prime Agent Worker

This reference architecture specifies the infrastructure-as-code configuration to deploy a containerized **Prime Agent** cognitive worker to AWS. The service executes as an autonomous, event-driven supervisor within ECS Fargate, decoupled from real-time edge computer vision streams via Amazon SQS and backed by Amazon EFS for persistent harness memory and session trees.

Architecture Overview

Component	AWS Primitive	Operational Role
Compute Engine	ECS on Fargate (Linux/ARM64 or x86_64)	Executes containerized daemon, Node.js supervisor, and headless IPython REPL.
Ingestion Broker	Amazon SQS + DLQ	Buffers tracklet anomalies, spatial triggers, and site alerts emitted from edge vision.
Persistent Storage	Amazon EFS + Access Point	Preserves /root/.prime/agent/, refined sub-agent specs, and RLM session memory across restarts.
Secrets & Governance	AWS Secrets Manager + IAM Roles	Injects LLM API tokens (OpenRouter/Z.ai/AWS Bedrock) into container memory at runtime.

1. Variables & Inputs (variables.tf)

```
variable "aws_region" {
  type      = string
  default   = "us-east-1"
  description = "AWS deployment region."
}
```

```

variable "environment" {
    type          = string
    default       = "production"
    description   = "Deployment stage tag (production/staging)."

```

2. Telemetry Queue & Dead Letter Queue (sqs.tf)

```

# Dead-letter queue for failed or corrupted reasoning payloads
resource "aws_sqs_queue" "agent_dlq" {
    name          =

```

```

"${var.project_name}-incident-dlq-${var.environment}"
  message_retention_seconds = 1209600 # 14 days

  tags = {
    Project      = var.project_name
    Environment  = var.environment
  }
}

# Main queue buffering anomaly events dispatched by the edge vision
filter
resource "aws_sqs_queue" "incident_queue" {
  name =
"${var.project_name}-incident-queue-${var.environment}"
  visibility_timeout_seconds = 300 # Set higher than max RLM execution
turn timeout
  message_retention_seconds = 86400

  redrive_policy = jsonencode({
    deadLetterTargetArn = aws_sqs_queue.agent_dlq.arn
    maxReceiveCount     = 3
  })

  tags = {
    Project      = var.project_name
    Environment  = var.environment
  }
}

```

3. Persistent Volume via Amazon EFS (efs.tf)

```

# Dedicated security group for EFS mount targets
resource "aws_security_group" "efs_sg" {
  name = "${var.project_name}-efs-sg-${var.environment}"
  description = "Allow inbound NFS from ECS Fargate Prime Agent tasks"
  vpc_id = var.vpc_id

  ingress {
    description = "NFS inbound from ECS Agent"
    from_port = 2048
    to_port = 2048
    protocol = "tcp"
    security_groups = [aws_security_group.agent_task_sg.id]
  }

  egress {

```

```

    from_port    = 0
    to_port      = 0
    protocol     = "-1"
    cidr_blocks  = ["0.0.0.0/0"]
  }

  tags = {
    Project      = var.project_name
    Environment  = var.environment
  }
}

# EFS Filesystem to store the agent's .prime state, refined specs, and
session logs
resource "aws_efs_file_system" "agent_storage" {
  creation_token = "${var.project_name}-efs-${var.environment}"
  encrypted      = true

  performance_mode = "generalPurpose"
  throughput_mode  = "bursting"

  lifecycle_policy {
    transition_to_ia = "AFTER_30_DAYS"
  }

  tags = {
    Name          = "${var.project_name}-efs-${var.environment}"
    Project       = var.project_name
    Environment    = var.environment
  }
}

# Mount targets created across all designated private subnets
resource "aws_efs_mount_target" "mount" {
  count          = length(var.private_subnet_ids)
  file_system_id = aws_efs_file_system.agent_storage.id
  subnet_id      = var.private_subnet_ids[count.index]
  security_groups = [aws_security_group.efs_sg.id]
}

# EFS Access Point to enforce POSIX permissions for the agent
execution root
resource "aws_efs_access_point" "agent_access_point" {
  file_system_id = aws_efs_file_system.agent_storage.id

  posix_user {
    gid = 1000
    uid = 1000
  }
}

```

```

    }

    root_directory {
      path = "/prime_harness"
      creation_info {
        owner_gid   = 1000
        owner_uid   = 1000
        permissions = "755"
      }
    }
  }

  tags = {
    Project      = var.project_name
    Environment = var.environment
  }
}

```

4. Security Groups & IAM Roles (security_iam.tf)

```

# Fargate Task Security Group
resource "aws_security_group" "agent_task_sg" {
  name          = "${var.project_name}-task-sg-${var.environment}"
  description   = "Egress control for Prime Agent container"
  vpc_id        = var.vpc_id

  egress {
    description = "Allow outbound HTTPS for LLM API providers and AWS
services"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    description = "Allow NFS communication to EFS mount targets"
    from_port   = 2049
    to_port     = 2049
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Project      = var.project_name
    Environment = var.environment
  }
}

```

```

}

# ECS Execution Role (Pulls images, fetches secrets, pushes CloudWatch
logs)
resource "aws_iam_role" "ecs_execution_role" {
  name = "${var.project_name}-ecs-execution-role-${var.environment}"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action      = "sts:AssumeRole"
      Effect      = "Allow"
      Principal = { Service = "ecs-tasks.amazonaws.com" }
    }]
  })
}

resource "aws_iam_role_policy_attachment" "ecs_execution_base" {
  role      = aws_iam_role.ecs_execution_role.name
  policy_arn =
"arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy"
}

# ECS Task Role (Runtime permissions granted to the Prime Agent Python
process)
resource "aws_iam_role" "ecs_task_role" {
  name = "${var.project_name}-ecs-task-role-${var.environment}"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action      = "sts:AssumeRole"
      Effect      = "Allow"
      Principal = { Service = "ecs-tasks.amazonaws.com" }
    }]
  })
}

# SQS read/delete permissions for the event loop
resource "aws_iam_policy" "agent_sqs_policy" {
  name      =
"${var.project_name}-agent-sqs-policy-${var.environment}"
  description = "Allows Prime Agent worker to consume incident
messages"

  policy = jsonencode({
    Version = "2012-10-17"

```

```

Statement = [
  {
    Effect = "Allow"
    Action = [
      "sqs:ReceiveMessage",
      "sqs:DeleteMessage",
      "sqs:GetQueueAttributes",
      "sqs:ChangeMessageVisibility"
    ]
    Resource = aws_sqs_queue.incident_queue.arn
  }
]
}))
}

resource "aws_iam_role_policy_attachment" "agent_sqs_attach" {
  role       = aws_iam_role.ecs_task_role.name
  policy_arn = aws_iam_policy.agent_sqs_policy.arn
}

```

5. ECS Cluster, Task Definition & Service (main.tf)

```

# CloudWatch Log Group
resource "aws_cloudwatch_log_group" "agent_logs" {
  name           =
  "/ecs/${var.project_name}-worker-${var.environment}"
  retention_in_days = 30

  tags = {
    Project      = var.project_name
    Environment  = var.environment
  }
}

# ECS Cluster
resource "aws_ecs_cluster" "cluster" {
  name = "${var.project_name}-cluster-${var.environment}"

  setting {
    name  = "containerInsights"
    value = "enabled"
  }
}

# ECS Task Definition with EFS Volume Binding
resource "aws_ecs_task_definition" "agent_task" {

```

```

family                = "${var.project_name}-worker"
network_mode          = "awsvpc"
requires_compatibilities = ["FARGATE"]
cpu                   = toString(var.cpu)
memory                = toString(var.memory)
execution_role_arn    = aws_iam_role.ecs_execution_role.arn
task_role_arn         = aws_iam_role.ecs_task_role.arn

volume {
  name = "prime_agent_state"

  efs_volume_configuration {
    file_system_id      = aws_efs_file_system.agent_storage.id
    transit_encryption   = "ENABLED"
    authorization_config {
      access_point_id = aws_efs_access_point.agent_access_point.id
      iam              = "DISABLED"
    }
  }
}

container_definitions = jsonencode([
  {
    name      = "prime-agent-worker"
    image     = var.agent_image_uri
    essential = true

    mountPoints = [
      {
        sourceVolume = "prime_agent_state"
        containerPath = "/root/.prime"
        readOnly      = false
      }
    ]

    environment = [
      { name = "SQS_QUEUE_URL", value =
aws_sqs_queue.incident_queue.url },
      { name = "AWS_DEFAULT_REGION", value = var.aws_region },
      { name = "PRIME_HEADLESS", value = "true" },
      { name = "PRIME_LOG_LEVEL", value = "INFO" }
    ]

    logConfiguration = {
      logDriver = "awslogs"
      options = {
        "awslogs-group" =
aws_cloudwatch_log_group.agent_logs.name

```

```

        "awslogs-region"          = var.aws_region
        "awslogs-stream-prefix" = "agent"
    }
}
}
})

tags = {
    Project      = var.project_name
    Environment = var.environment
}
}

# ECS Fargate Service (Desired count = 1, can be scaled with SQS queue
depth)
resource "aws_ecs_service" "agent_service" {
    name                = "${var.project_name}-service-${var.environment}"
    cluster             = aws_ecs_cluster.cluster.id
    task_definition     = aws_ecs_task_definition.agent_task.arn
    desired_count       = 1
    launch_type         = "FARGATE"

    network_configuration {
        subnets          = var.private_subnet_ids
        security_groups    = [aws_security_group.agent_task_sg.id]
        assign_public_ip   = false
    }

    depends_on = [
        aws_efs_mount_target.mount
    ]

    tags = {
        Project      = var.project_name
        Environment = var.environment
    }
}

```

6. Outputs (outputs.tf)

```

output "incident_queue_url" {
    value          = aws_sqs_queue.incident_queue.url
    description    = "SQS URL to inject into edge video filter publishers."
}

output "incident_queue_arn" {

```

```
    value      = aws_sqs_queue.incident_queue.arn
    description = "ARN of the incident processing queue."
}

output "efs_file_system_id" {
    value      = aws_efs_file_system.agent_storage.id
    description = "ID of the shared persistent volume."
}

output "ecs_service_name" {
    value      = aws_ecs_service.agent_service.name
    description = "Name of the deployed Fargate ECS service."
}
```